"Employee Management System"

Submitted By

Student Name	Student ID
Sharmin Akter Jame	232-15-380
Tanvir Hossan Jisan	232-15-870
A.Q Mohiuddin Mahim	232-15-869
Al Zubair Ahmed	232-15-591
Foysal Mahamud Fahim	232-15-334

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE222: Object-Oriented Programming Lab in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 13, 2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **MD. Jubayar Alam Rafi**, **Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

MD. Jubayar Alam Rafi
Lecturer
Department of Computer Science and Engineering Daffodil
International University

Submitted by

Sharmin Akter Jame ID:232-15-380 Dept. of CSE, DIU	
Tanvir Hossan Jisan ID:232-15-870 Dept. of CSE, DIU	A.Q Mohiuddin Mahim ID:232-15-869 Dept. of CSE, DIU
Al Zubair Ahmed ID:232-15-591 Dept. of CSE, DIU	Foysal Mahamud Fahim ID:232-15-334 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand and classify key OOP concepts, including interfaces and abstract classes.
CO2	Apply OOP principles to design and implement Java-based solutions for real-world problems.
CO3	Use UML diagrams to design and communicate software solutions.
CO4	Develop advanced Java applications using exception handling, multithreading, and networking
CO5	Solve a Complex Engineering Problem (CEP) through a comprehensive course project integrating OOP concepts, UML, and modern tools

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Declaration i

Course & Program Outcome ii

1 Introduction **1-3**

1.1 Introduction 1
1.2 Motivation 1
1.3 Objectives 1
1.4 Feasibility Study 2
1.5 Gap Analysis 2
1.6 Project Outcome 3

2 Proposed Methodology/Architecture **4-13**

2.1 Requirement Analysis & Design Specification 4
2.1.1 Overview 4
2.1.2 Proposed Methodology/ System Design 5
2.1.3 UI Design 8
2.2 Overall Project Plan 13

3 Implementation and Results **14-15**

3.1 Implementation 14
3.2 Performance Analysis 14
3.3 Results and Discussion 14

4 Engineering Standards and Mapping **16-19**

4.1 Impact on Society, Environment and Sustainability 16
4.1.1 Impact on Life 16
4.1.2 Impact on Society & Environment 16
4.1.3 Ethical Aspects 16
4.1.4 Sustainability Plan 16
4.2 Project Management and Team Work 17
4.3 Complex Engineering Problem 18
4.3.1 Mapping of Program Outcome 18
4.3.2 Complex Problem Solving 18
4.3.3 Engineering Activities 19

5 Conclusion **20-21**
5.1 Summary..20
5.2 Limitation..20
5.3 Future Work..21

References **22**

Chapter 1

Introduction

1.1 Introduction

The Employee Management System (EMS) is a Java-based desktop application designed to manage employee records within an organization. The application uses Object-Oriented Programming (OOP) principles to handle employee data securely and efficiently. It features modules for admin & employee recording, viewing, and management.

1.2 Motivation

In any organization, managing employee information manually can be time-consuming, error-prone, and inefficient. The need for a digital solution to store, update, and retrieve employee data in a structured and secure manner motivated the development of this project.

As students of Object-Oriented Programming (OOP), we aimed to apply the theoretical concepts we learned—such as encapsulation, inheritance, abstraction, and polymorphism—in a real-world application. Building an Employee Management System allowed us to:

- Strengthen our understanding of Java programming.
- Explore GUI development using Swing.
- Learn how to connect applications with a database using JDBC.
- Solve a real-world problem through clean, modular code.

1.3 Objectives

Object-Oriented Programming (OOP) is the foundation of the Employee Management System. The goal of this system is to effectively handle a number of employee-related tasks, employee information handling and so on.

This project will concentrate on putting important OOP ideas like encapsulation, inheritance, polymorphism, and abstraction into practice in order to guarantee modularity, scalability, and maintainability.

1.4 Feasibility Study

A feasibility study was conducted to determine whether the Employee Management System could be developed and implemented successfully with the available resources, time, and technology. The study included the following types of feasibility:

- **Technical Feasibility:** The project is technically feasible as it uses standard and widely supported technologies such as Java, Swing, and MySQL. All the development tools and libraries are open-source or freely available. The development environment can run efficiently on commonly available hardware.
- **Operational Feasibility:** The system is user-friendly and requires minimal training to operate. It simplifies routine Admin tasks such as adding, viewing, updating, removing, checking and managing employee records etc. The use of a graphical user interface (GUI) makes it accessible even for users with limited technical knowledge.
- **Economic Feasibility**: There are no significant financial costs involved in the development of this project. All software tools used (Java, IDE, MySQL) are open-source. The cost of implementation is minimal, making the project economically viable for small to medium organizations.
- **Schedule Feasibility:** The project was planned and developed within the given academic timeline. The development phases—including planning, design, coding, and testing—were completed on schedule, ensuring timely delivery.

1.5 Gap Analysis

The current employee management process is mostly manual, involving spreadsheets or paper records, which is inefficient, error-prone, and lacks security.

The Employee Management System addresses these gaps by:

- Automating record management through a database.
- Providing a user-friendly GUI for interaction.
- Adding login-based security.
- Ensuring faster access and updates.
- Offering scalability for future enhancements.

1.6 Project Outcome

The Employee Management System was successfully developed using Java and Object-Oriented Programming principles. It provides a simple and effective solution for managing employee records with features like:

- Secure user login
- Add, view, update, and delete employee details
- Manage leaves, manage pay-slip, checking attendance, managing notice
- Graphical User Interface (GUI) for ease of use
- Real-time database connectivity using MySQL

Chapter 2

Proposed Methodology/Architecture

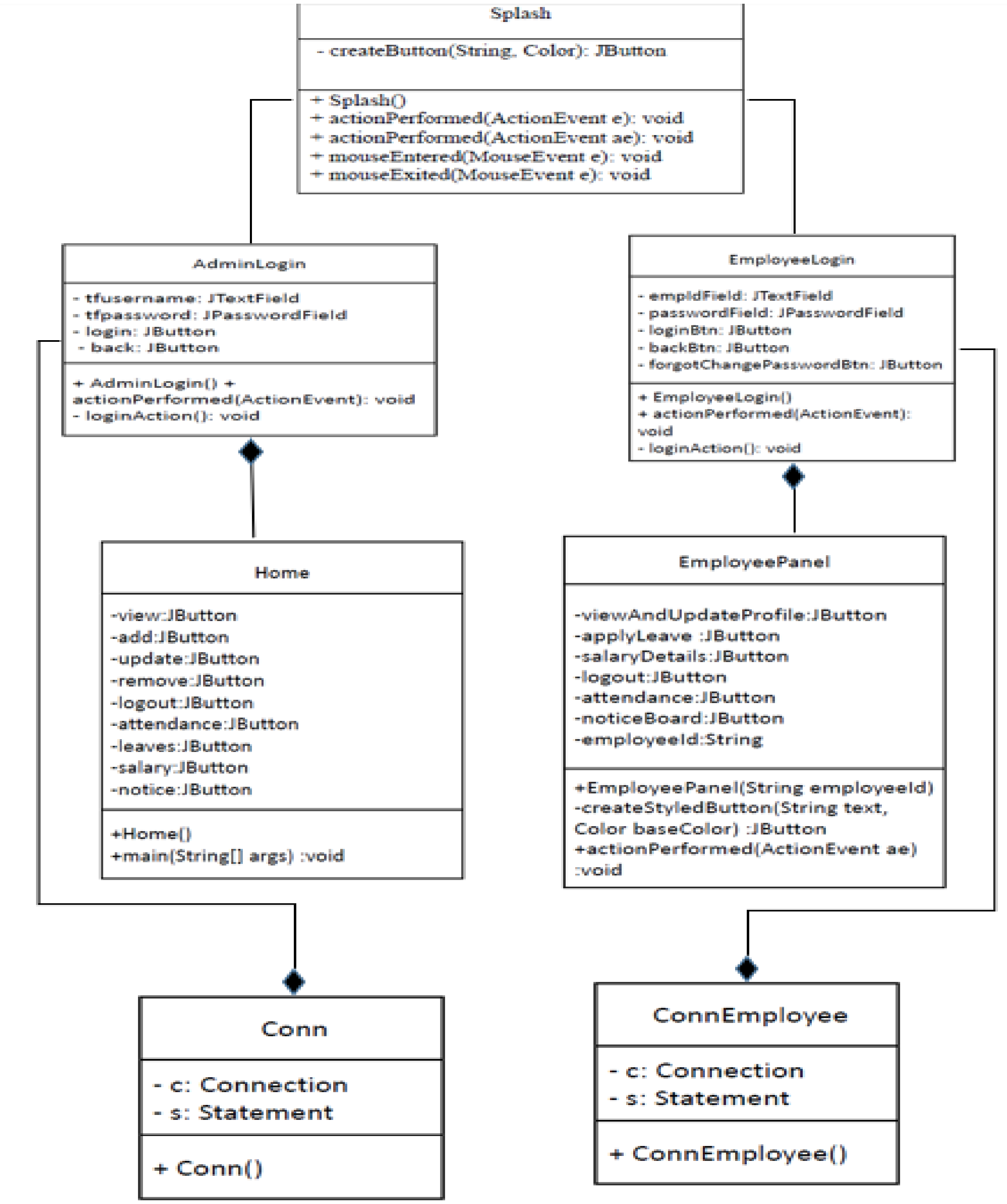
2.1 Requirement Analysis & Design Specification

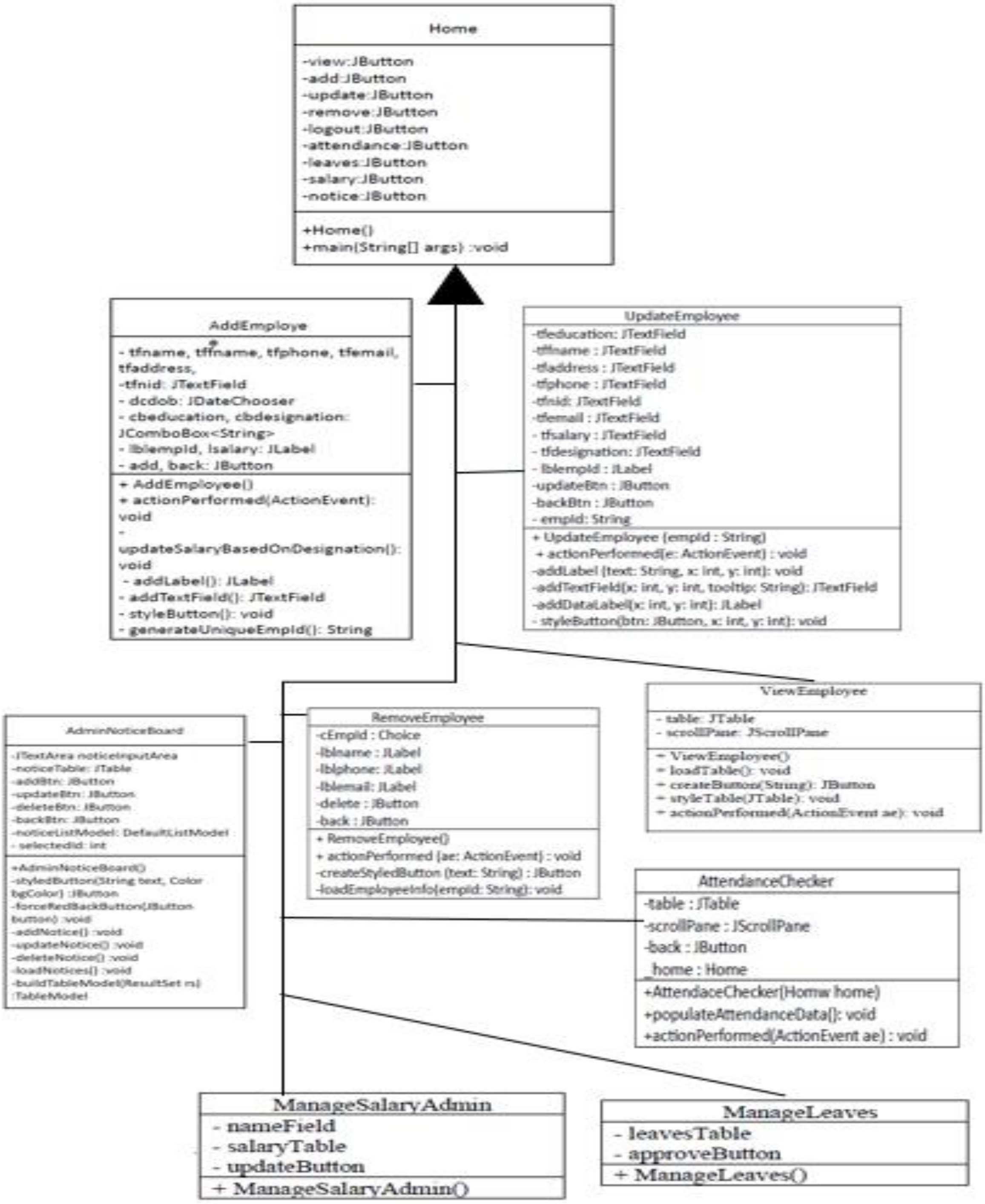
2.1.1 Overview

The Employee Management System is a Java-based desktop application designed to manage employee information efficiently. It allows Admin to perform operations such as adding, viewing, updating, and deleting, manage leaves, manage pay-slip, checking attendance, manageing notice and manage employee records through a graphical user interface (GUI) built with Java Swing. The system connects to a MySQL database using JDBC for real-time data management.

This project follows Object-Oriented Programming (OOP) principles to ensure code modularity, reusability, and scalability. It is intended to simplify HR tasks and improve the accuracy and accessibility of employee data in small to medium-sized organizations.

2.1.2 Proposed Methodology/ System Design





Home
-view:JButton
-add:JButton
-update:JButton
-remove:JButton
-logout:JButton
-attendance:JButton
-leaves:JButton
-salary:JButton
-notice:JButton
+Home()
+main(String[] args) :void
AddEmploye
- tfname, tffname, tfphone, tfemail, tfaddress,
-tfnid: JTextField
- dcdob: JDateChooser
- cbeducation, cbdesignation: JComboBox<String>
- lblempId, lsalary: JLabel
- add, back: JButton
+ AddEmployee()
+ actionPerformed(ActionEvent): void
- updateSalaryBasedOnDesignation(): void
- addLabel(): JLabel
- addTextField(): JTextField
- styleButton(): void
- generateUniqueEmpId(): String
UpdateEmployee
-tfeducation: JTextField
-tffname : JTextField
-tfaddress : JTextField
-tfphone : JTextField
-tfnid: JTextField
-tfemail : JTextField
- tfsalary : JTextField
- tfdesignation: JTextField
- lblempId : JLabel
-updateBtn : JButton
-backBtn : JButton
- empId: String
+ UpdateEmployee (empId : String)
+ actionPerformed(e: ActionEvent) : void
-addLabel (text: String, x: int, y: int): void
-addTextField(x: int, y: int, tooltip: String): JTextField
-addDataLabel(x: int, y: int): JLabel
- styleButton(btn: JButton, x: int, y: int): void
AdminNoticeBoard
-JTextArea noticeInputArea
-noticeTable: JTable
-addBtn: JButton
-updateBtn: JButton
-deleteBtn: JButton
-backBtn: JButton
-noticeListModel: DefaultListModel
- selectedId: int
+AdminNoticeBoard()
-styledButton(String text, Color bgColor) :JButton
-forceRedBackButton(JButton button) :void
-addNotice() :void
-updateNotice() :void
-deleteNotice() :void
-loadNotices() :void
-buildTableModel(ResultSet rs) :TableModel
RemoveEmployee
-cEmpId : Choice
-lblname : JLabel
-lblphone: JLabel
-lblemail: JLabel
-delete : JButton
-back : JButton
+ RemoveEmployee()
+ actionPerformed (ae: ActionEvent) : void
-createStyledButton (text: String) : JButton
-loadEmployeeInfo(empId: String): void
ViewEmployee
- table: JTable
- scrollPane: JScrollPane
+ ViewEmployee()
- loadTable(): void
- createButton(String): JButton
- styleTable(JTable): void
+ actionPerformed(ActionEvent ae): void
AttendanceChecker
-table : JTable
-scrollPane : JScrollPane
-back : JButton
_home : Home
+AttendaceChecker(Homw home)
+populateAttendanceData(): void
+actionPerformed(ActionEvent ae) : void
ManageSalaryAdmin
- nameField
- salaryTable
- updateButton
+ ManageSalaryAdmin()
ManageLeaves
- leavesTable
- approveButton
+ ManageLeaves()

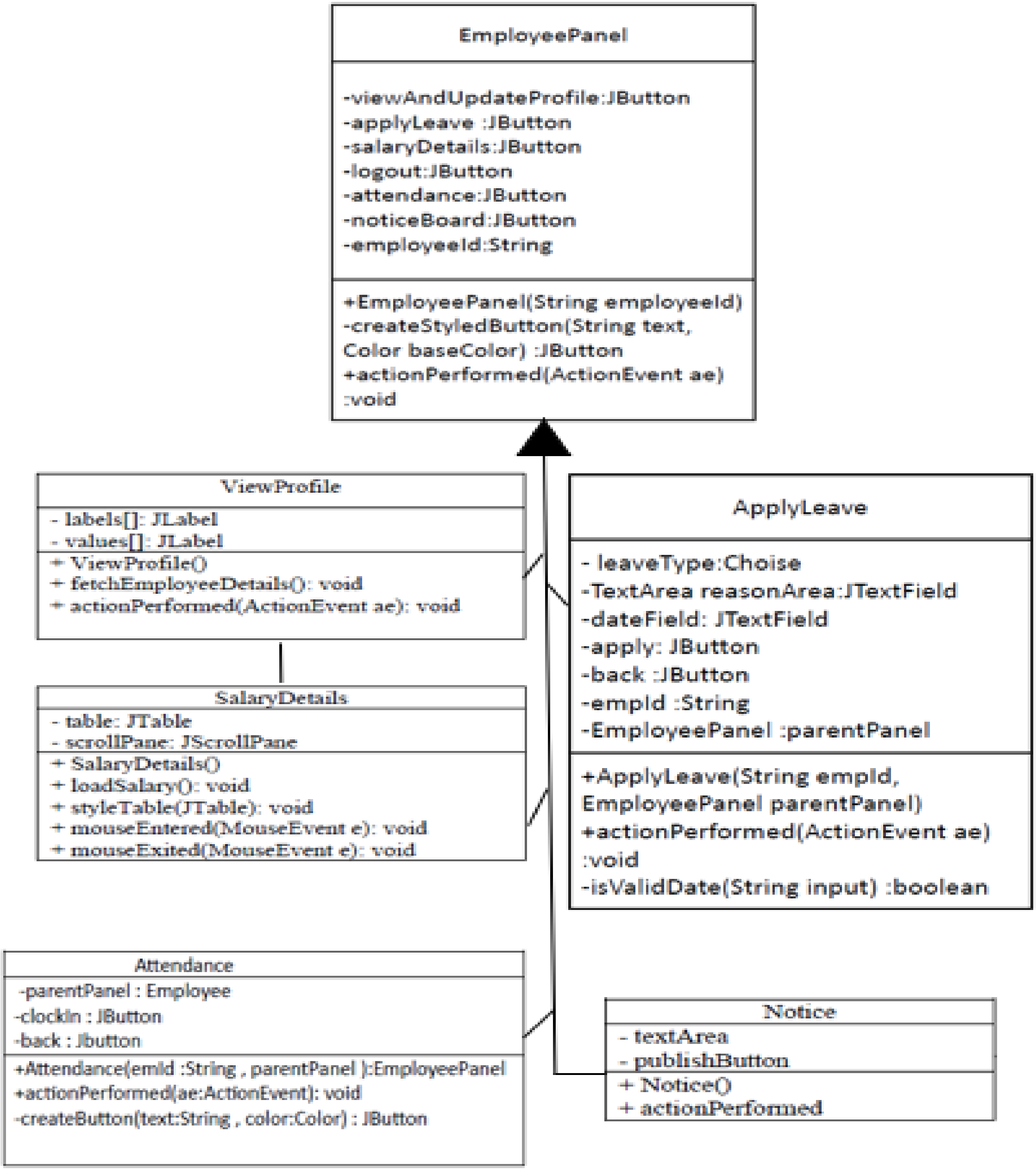
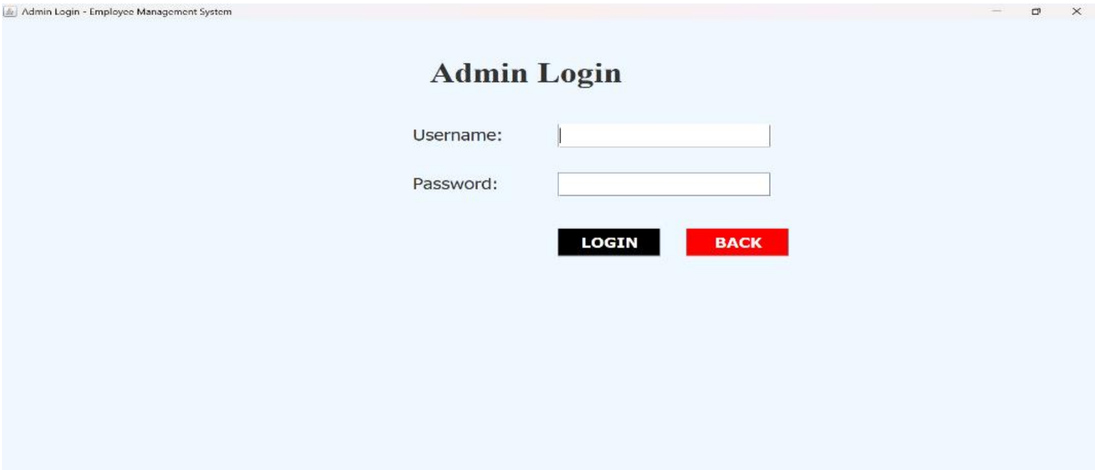


Figure 2.1: This is a sample diagram

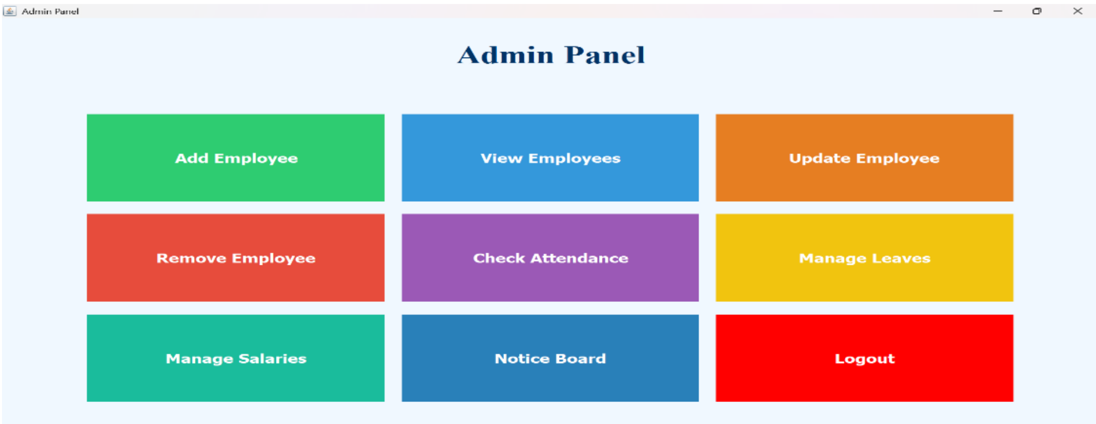
2.1.3 UI Design:

Splash

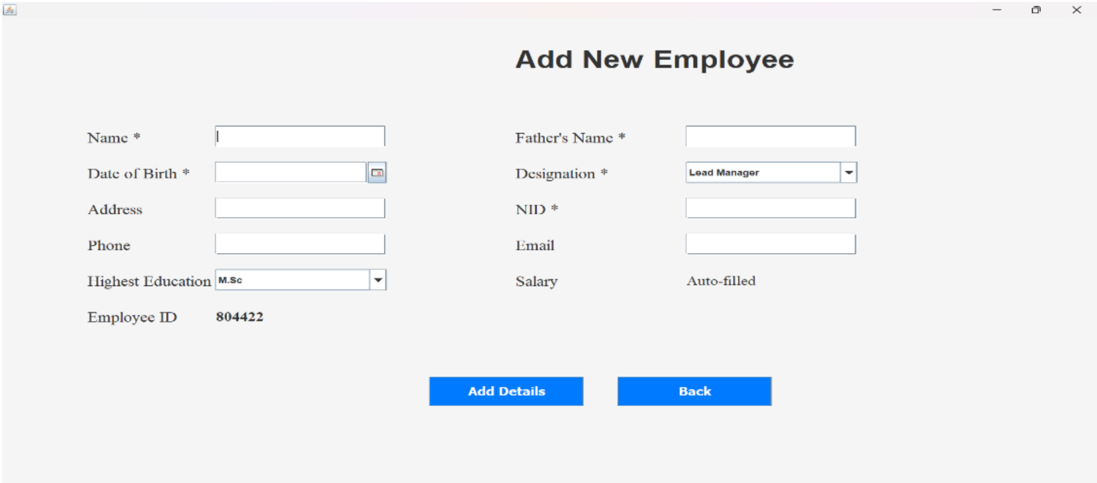
Admin Login Panel



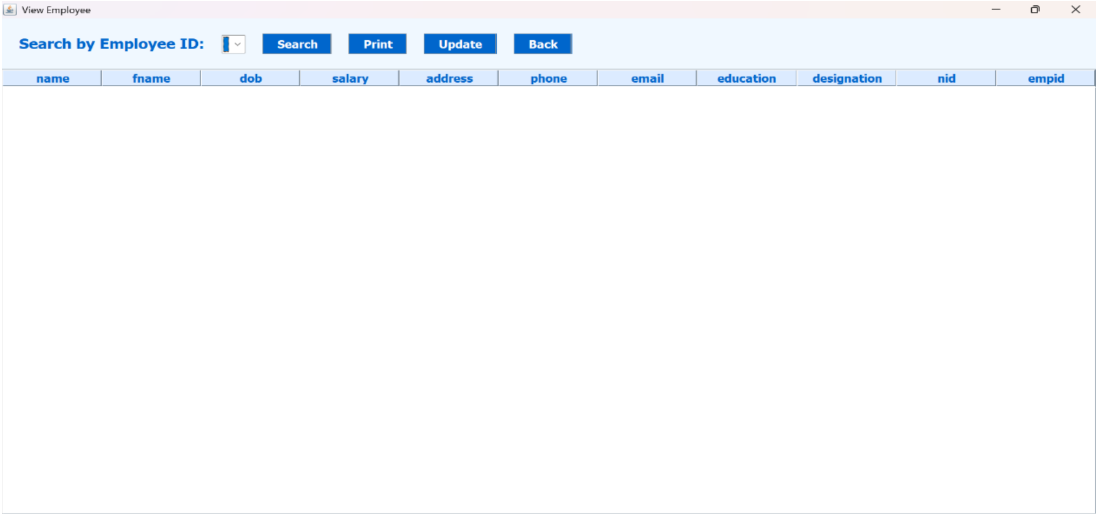
Admin Panel



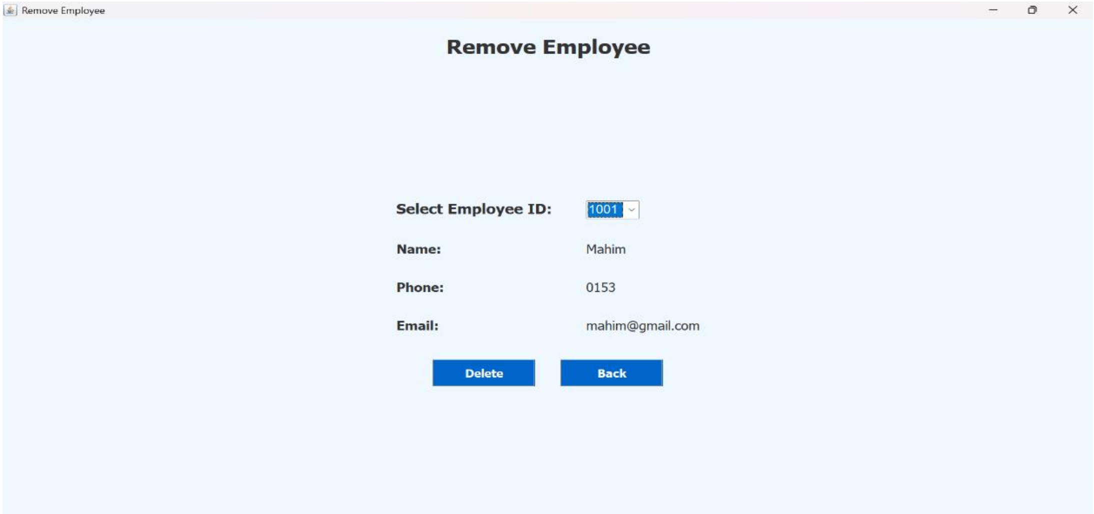
Add Employee



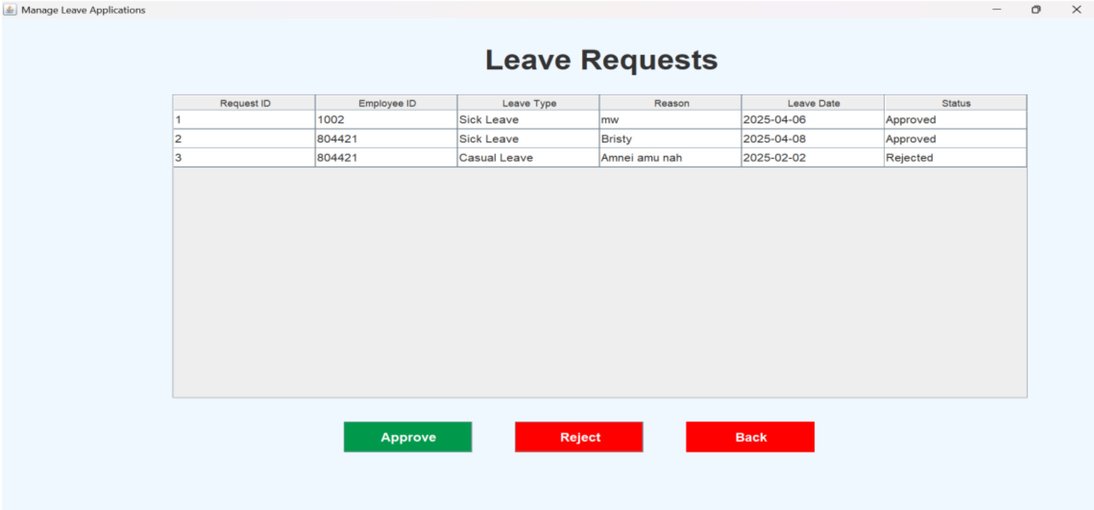
View Employee



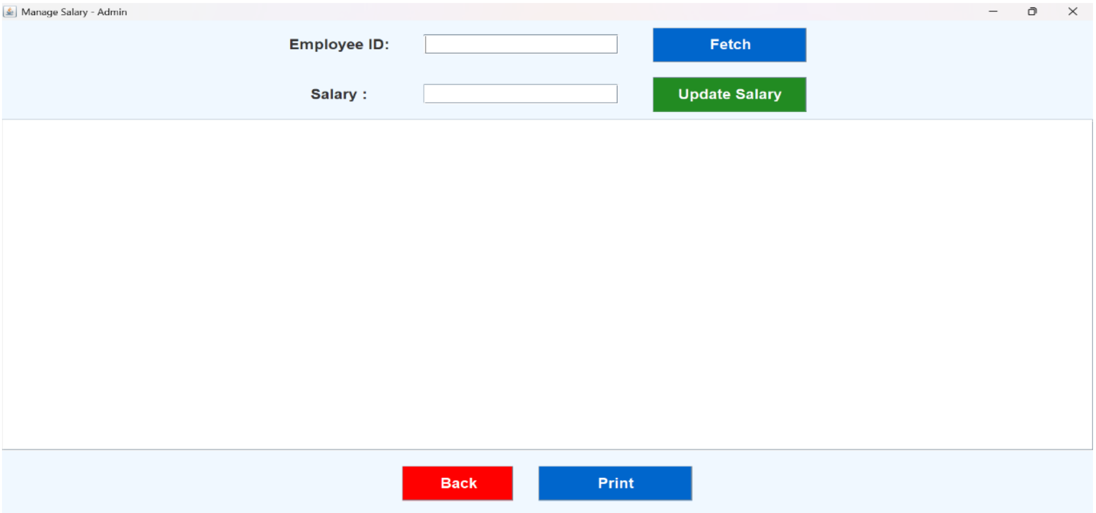
Remove Employee Info



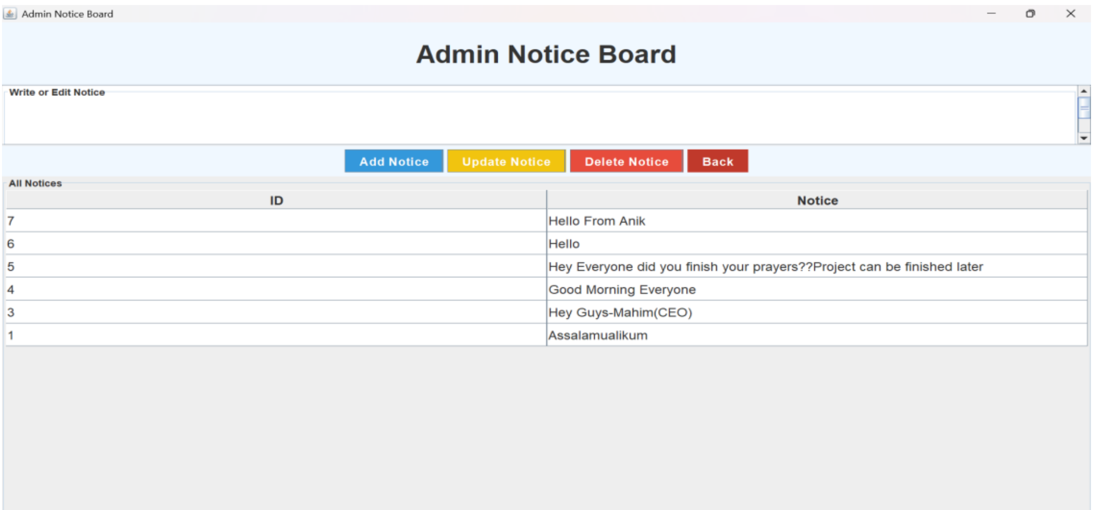
Manage Leave



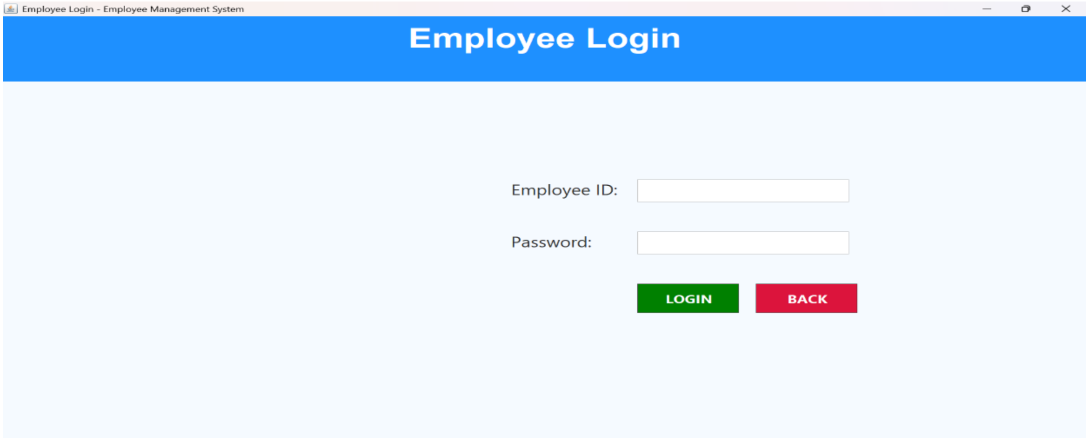
Manage Salary



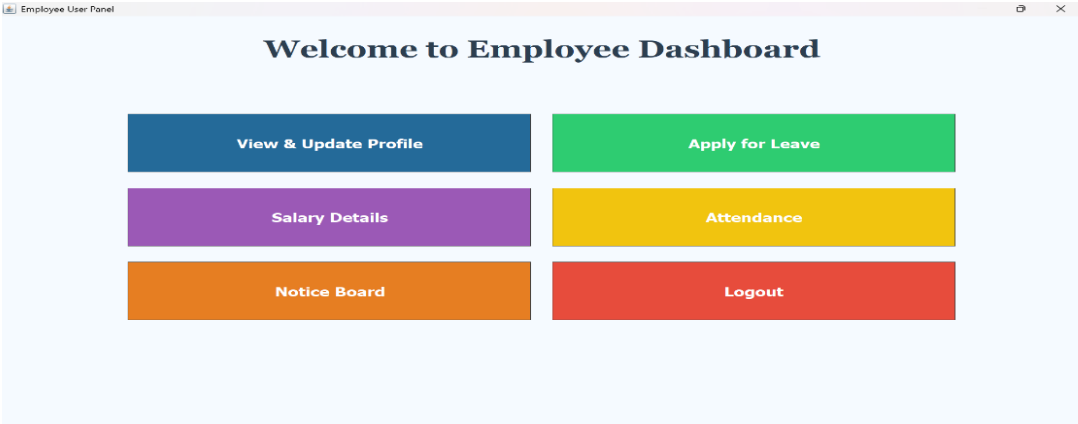
Admin Notice Panel



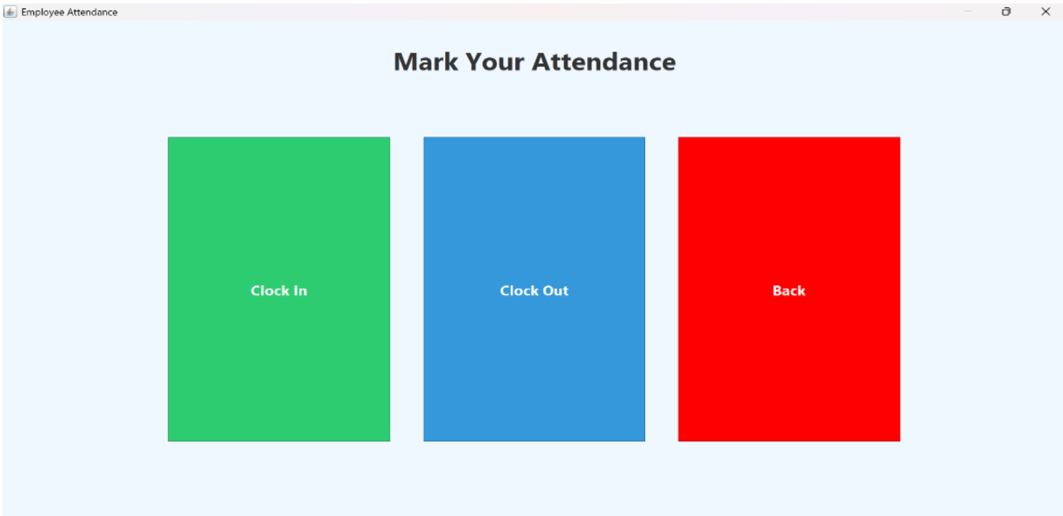
Employee Login



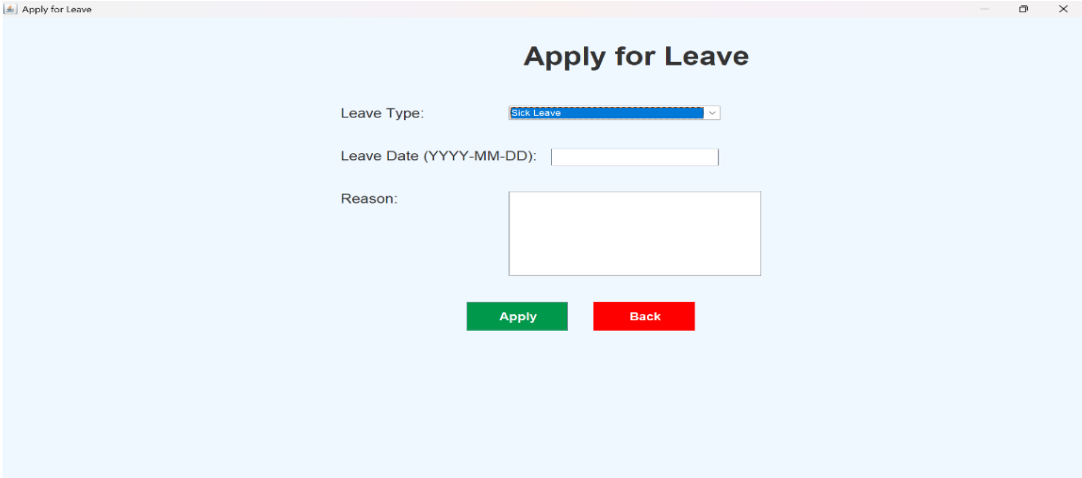
Employee Panel



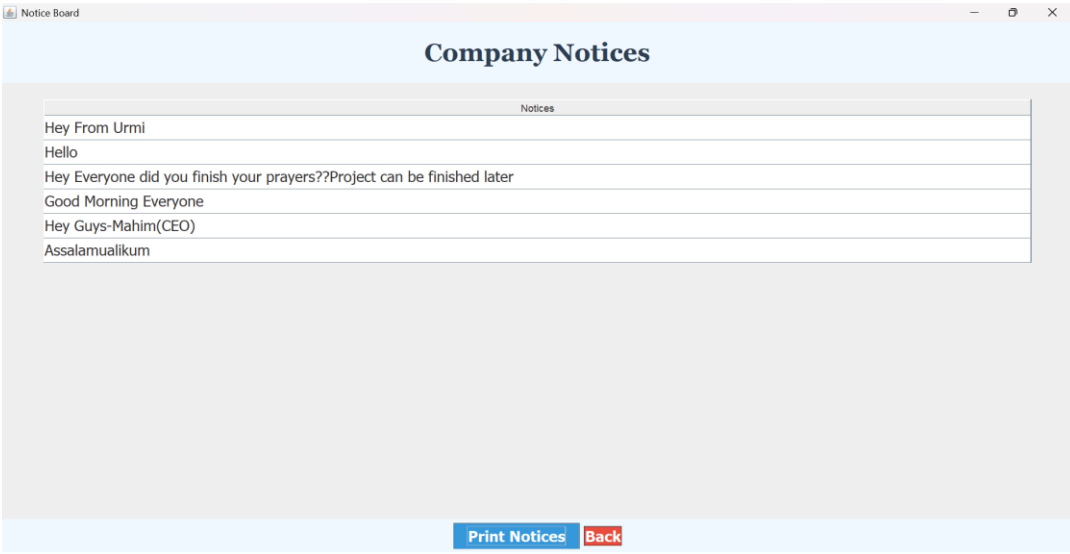
Attandance



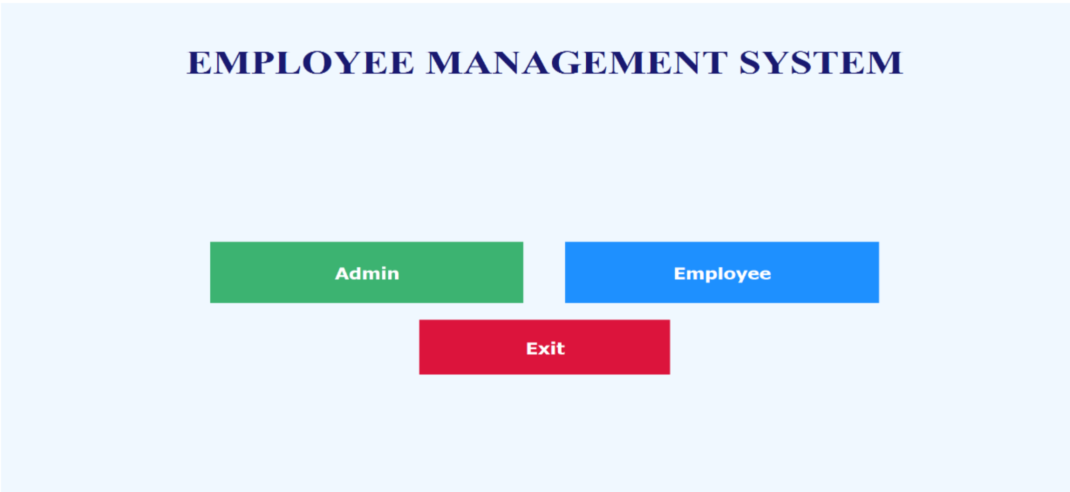
Apply Leave



Notice Board



Exit



2.2 Overall Project Plan

The development of the **Employee Management System** was divided into several key phases to ensure smooth progress and timely completion. Each phase focused on specific objectives aligned with the project timeline.

The project was completed in five main phases:

- **Requirement Analysis** – Identified system features and user needs.
- **Design** – Planned UI with Swing and database structure in MySQL.
- **Implementation** – Developed modules using Java and OOP concepts.
- **Testing** – Checked each module for errors and fixed bugs.
- **Documentation** – Prepared the final report and user guide.

This plan ensured organized development and timely completion of the project.

Chapter 3

Implementation and Result

3.1 Implementation

The system was implemented using Java for the application logic and Swing for the graphical user interface. MySQL was used as the backend database, and JDBC was used for connectivity. The project follows OOP principles, with each functionality handled by separate Java classes such as AdminLogin.java, Conn.java, AddEmployee.java, Remove.java, View.java, Update.java, AdminNoticeBoard.java, ApplyLeave.java, Attendance.java, AttendanceChecker.java, ConnEmplpoyee.java, EmployeeLogin.java, EmployeePanel.java, Home.java, ManageLeaves.java, ManageSalaryAdmin.java, Notice.java, SalaryDetails.java, ViewProfile.java etc. The system includes modules for login, employee record management and database handling.

3.2 Performance Analysis

The application performs efficiently with a smooth GUI and quick response time. Basic performance observations:

- **Login Time:** Less than 2 seconds.
- **Database Operations:** Completed in real-time with no noticeable delay.
- **Memory Usage:** Low, as the app runs locally with lightweight modules.
- **Stability:** Stable for basic operations even after multiple transactions.

Overall, the system performs well for small to medium data loads typical in educational or small business environments

3.3 Results and Discussion

The system met all its functional requirements:

- Successful login authentication.
- Accurate CRUD operations on employee data.
- User-friendly interface with minimal learning curve.

The modular structure and use of OOP concepts made the code clean and manageable. The project demonstrated practical knowledge of Java, GUI design, and database integration. Future enhancements like role-based access or cloud integration can make it even more powerful

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The Employee Management System simplifies Admin & Employee tasks by automating employee record-keeping, reducing manual errors, and saving time. It improves productivity for individuals working in administrative roles, making day-to-day management faster and more efficient.

4.1.2 Impact on Society & Environment

By digitalizing employee records, the system reduces the need for paper-based documentation, contributing to environmental sustainability. It promotes digital transformation in small to mid-sized businesses, helping them operate more efficiently and reduce operational costs.

4.1.3 Ethical Aspects

The system includes a login feature to protect employee data, promoting data privacy and responsible usage. However, future improvements like encrypted passwords and access control can further enhance ethical data handling and prevent unauthorized access.

4.1.4 Sustainability Plan

The system is designed to be scalable and maintainable. It can be updated with new features like role-based access, cloud storage, or report generation without overhauling the entire structure. By using open-source tools (Java, MySQL), the system remains cost-effective and sustainable in the long run.

4.2 Project Management and Team Work

Team Structure and Roles:

Date	Task/Activity	Assigned Team Member	Status	Comments/Remarks
March 10, 2025	**Project Planning and Research**	All Team Members	Completed	Initial research, UML, project objectives.
March 16, 2025	**Project Proposal**	Foysal Mahamud Fahim, A.Q Mohiuddin Mahim	Completed	Accepted “Employee Management System”
March 18, 2025	**UML design**	Sharmin Akter Jame	Completed	UML, schematic drawn.
March 24, 2025	**UI design**	Al Zubair Ahmed, Tanvir Hossan Jisan	Completed	UI, schematic drawn.
March 25, 2025	**Code**	All Team Members	Completed	Start Coding.
March 30, 2025	**LoginEmp, ManageLeaves, ManageSalary, Notice**	Foysal Mahamud Fahim	Completed	Initial testing phase.
March 30, 2025	**Splash,ViewEmployee,viewProfile SalaryDetalis.**	Sharmin Akter Jame,	Completed	Initial testing phase.
March 30, 2025	**Home, AdminNoticeBoard, ApplyLeave, EmployeePanel,**	Tanvir Hossan Jisan	Completed	Initial testing phase.
March 30, 2025	**Conn, ConnEmployee, AdminLogin, EmployeeLogin, AddEmployee**	A.Q Mohiuddin Mahim	Completed	Initial testing phase.
March 30, 2025	**UpdateEmployee, RemoveEmployee, Attendance, AttendanceChecker**	Al Zubair Ahmed	Completed	Initial testing phase.
April 09, 2025	**Final Code Selection and Edits**	A.Q Mohiuddin Mahim	Completed	Fixed all Debug.
April 16, 2025	**Final Review**	All Members	Completed	Final checks.
April 10, 2025	**Final Report Writing and Edits**	Foysal Mahamud Fahim	Completed	Write report and make edits.
April 13, 2025	**Final Review and Submission**	All Team Members.	Completed	Final checks and submission of project.

Time and Cost:
Overall, time needed is **30-35 days**, and the total cost of this project is **10k**.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

Table 4.1: Justification of Program Outcomes

PO's	**Justification**
PO1	Apply engineering knowledge to computing problems.
PO2	Analyze and solve engineering problems.
PO3	Design and develop software solutions.
PO5	Use modern tools for software development.
PO9	Work effectively as a team member or leader.
PO12	Engage in lifelong learning.

4.3.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
CO1	**CO1, CO2**	**CO2**	**CO3, CO4**			

4.3.3 Engineering Activities

The development of the **Employee Management System** involved solving several complex problems that required a structured and logical approach. These included:

- **Database Integration**: Establishing smooth and secure communication between the Java application and MySQL using JDBC.
- **Modular Design Using OOP**: Breaking down the system into manageable modules while ensuring code reusability, maintainability, and minimal redundancy.
- **Handling Real-Time Data Operations**: Ensuring accurate and instant execution of CRUD operations without data loss or inconsistency.
- **GUI and Backend Coordination**: Synchronizing the front-end user interface with backend logic to provide a seamless user experience.
- **Error Handling**: Managing unexpected user input or system failures using proper exception handling to prevent crashes.

By addressing these challenges, the project enhanced skills in logical thinking, debugging, and system design—core aspects of complex problem solving in software development.

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity
CO3, CO4	**CO4**			

Chapter 5

Conclusion

5.1 Summary

The **Employee Management System** was successfully developed using Java and MySQL, applying core Object-Oriented Programming principles. It allows secure login and efficient management of employee data through a user-friendly interface. The system meets its primary goals of automating employee record handling and improving administrative efficiency.

5.2 Limitation

Despite meeting its core objectives, the **Employee Management System** has some limitations

- **No Password Recovery**: Credentials are stored or verified without encryption, which poses a security risk.
- **Local-Only Access**: The system runs only on a local machine and cannot be accessed remotely or via the internet.
- **No Backup or Recovery Option**: There is no automatic backup system to recover data in case of corruption or failure.
- **Not Scalable for Large Organizations**: The design is best suited for small-scale use and may struggle with thousands of records.
- **Local Storage Only**: The system is limited to a local machine and does not support remote or cloud-based access.
- **Limited UI Experience**: GUI design is basic and not optimized for modern user experience.
- **Manual Attendance:** Attendance is not tracked or recorded automatically.

5.3 Future Work

To enhance the functionality and relevance of the system in modern workplaces, the following future developments are proposed:

1. **Password Reset System:** User can change their password for security reasons or if they forget Password.
2. **Automatic Attendance System**: Integration with biometric, RFID, or facial recognition systems for real-time attendance logging.
3. **Cloud-Based Deployment**: Hosting the system on cloud platforms for global access, data sync, and scalability.
4. **AI-Powered Analytics**: Implementing AI to analyze employee data, generate insights, predict trends, and support HR decision-making.
5. **Mobile Application**: Developing an Android/iOS app for mobile access to records, attendance, and notifications.
6. **Data Visualization & Reports**: Creating dynamic dashboards and visual reports for better data representation and understanding.
7. **Notification System**: Adding SMS, email, or in-app alerts for attendance status, profile updates, or HR announcements.

References

1. Books:
    - "Java: The Complete Reference" **by Herbert Schildt.**
    - "Database Systems: The Complete Book" **by Hector Garcia-Molina, Jeff Ullman, and Jennifer Widom**
2. Research Papers and Articles:
    - **Security and Privacy in Java-Based Applications":** This article discusses security measures, like encryption and secure login systems, which can be useful for improving your project's security.
3. Online Resources:
    - https://docs.oracle.com/javase/8/docs/
    - https://dev.mysql.com/doc/
    - https://google.com
    - https://chatgpt.com
    - https://deepseekai.com
    - https://programmiz.com/java/oop
    - https://elearn.daffodilvarsity.edu.bd/